

Name - Jayesh Deshmukh
Roll Number - 57
Class - D15C

EXPERIMENT 2 : Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets

1. Dataset Source:

Dataset Name: Students Performance Dataset

Source: Kaggle

Link: <https://www.kaggle.com/datasets/spscientist/students-performance-in-exams>

The dataset contains student demographic and academic performance features used for predicting exam scores using regression techniques.

2. Dataset Description:

The Students Performance dataset is used to demonstrate Multiple Linear Regression, Ridge Regression, and Lasso Regression for comparison purposes.

This dataset contains student-based numerical and categorical features extracted from demographic details and subject scores.

Although Linear Regression is generally used for continuous prediction problems, here it is applied to predict students' math scores.

- Size: 1000 samples (rows) × 8 attributes (columns)
- Target Variable:
math score (Continuous variable ranging from 0–100)
- Input Features:
 1. gender
 2. race/ethnicity
 3. parental level of education
 4. lunch
 5. test preparation course
 6. reading score

- Dataset Characteristics:
 - Mix of categorical and numerical features
 - No major missing values
 - Suitable for regression analysis
 - Ideal for comparing regularization techniques

3. Mathematical Formulation:

Step 1: Linear Combination

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Where:

w = weights

x = feature values

y = predicted output (math score)

Step 2: Cost Function (Mean Squared Error)

$$J(w) = (1/m) \sum (y - \hat{y})^2$$

Ridge Regression (L2 Regularization):

$$J(w) = (1/m) \sum (y - \hat{y})^2 + \lambda \sum w^2$$

- Controls coefficient magnitude
- Reduces overfitting
- Handles multicollinearity

Lasso Regression (L1 Regularization):

$$J(w) = (1/m) \sum (y - \hat{y})^2 + \lambda \sum |w|$$

- Performs feature selection
- Can eliminate irrelevant variables
- Produces sparse model

4. Algorithm Limitations:

1. Assumes linear relationship between independent and dependent variables.

2. Sensitive to multicollinearity.
3. Outliers can significantly affect model performance.
4. Ridge cannot remove irrelevant features completely.
5. Lasso may remove important correlated features.

Not suitable when:

- Data has strong non-linear relationships.
- Dataset size is extremely small.

5. Methodology / Workflow:

Step 1: Data Collection

- Dataset downloaded from Kaggle.

Step 2: Data Preprocessing

- Load dataset using Pandas
- Check for null values
- Convert categorical variables using OneHotEncoding
- Separate features and target variable
- Perform train-test split (80% training, 20% testing)
- Apply StandardScaler for normalization

Step 3: Model Training

- Train Multiple Linear Regression model
- Train Ridge Regression model
- Train Lasso Regression model

Step 4: Model Evaluation

- Predict on test data
- Calculate MAE
- Calculate MSE
- Calculate R^2 Score

6. Performance Analysis:

Evaluation Metrics Used:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- R^2 Score

Typical Results:

Linear Regression:

$R^2 \approx 0.87$

Ridge Regression:

$R^2 \approx 0.88$

Lasso Regression:

$R^2 \approx 0.86$

Interpretation:

- Ridge slightly improves generalization.
- Lasso reduces model complexity.
- Regularization prevents overfitting.

7. Hyperparameter Tuning:

Hyperparameters Tuned:

Ridge:

- alpha (regularization strength)

Lasso:

- alpha (regularization strength)

Tuning Method:

GridSearchCV

Example:

Tested alpha values = [0.01, 0.1, 1, 10, 100]

Impact of Tuning:

Before tuning:

$R^2 \approx 0.86$

After tuning:

$R^2 \approx 0.88$

- Reduced overfitting
- Improved generalization

Code & Output

```
# =====  
# Multi Regression, Ridge, Lasso  
# =====
```

```

import pandas as pd
import numpy as np
import os
import requests
import io

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

# -----
# 1. Load Dataset
# -----
file_path = '/content/StudentsPerformance.csv'

# Directly load the file as it is available in the kernel
df = pd.read_csv(file_path)

print("First 5 rows:")
print(df.head())

# -----
# 2. Define Features and Target
# -----
# Example: Predict math score
X = df.drop("math score", axis=1)
y = df["math score"]

# -----
# 3. Handle Categorical Features
# -----
categorical_cols = X.select_dtypes(include=["object"]).columns
numerical_cols = X.select_dtypes(include=["int64", "float64"]).columns

preprocessor = ColumnTransformer(

```

```

transformers=[
    ("cat", OneHotEncoder(drop="first"), categorical_cols)
],
remainder="passthrough"
)

# -----
# 4. Train-Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# =====
# 5. Multiple Linear Regression
# =====
linear_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LinearRegression())
])

linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)

# =====
# 6. Ridge Regression
# =====
ridge_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", Ridge(alpha=1.0))
])

ridge_model.fit(X_train, y_train)
y_pred_ridge = ridge_model.predict(X_test)

# =====
# 7. Lasso Regression
# =====
lasso_model = Pipeline(steps=[
    ("preprocessor", preprocessor),

```

```

        ("model", Lasso(alpha=0.1))
    ])

lasso_model.fit(X_train, y_train)
y_pred_lasso = lasso_model.predict(X_test)

# -----
# 8. Evaluation Function
# -----
def evaluate_model(name, y_test, y_pred):
    print(f"\n{name} Results:")
    print("MAE :", mean_absolute_error(y_test, y_pred))
    print("MSE :", mean_squared_error(y_test, y_pred))
    print("R2  :", r2_score(y_test, y_pred))

# -----
# 9. Compare Models
# -----
evaluate_model("Multiple Linear Regression", y_test, y_pred_linear)
evaluate_model("Ridge Regression", y_test, y_pred_ridge)
evaluate_model("Lasso Regression", y_test, y_pred_lasso)

```

```

First 5 rows:
...
  gender race/ethnicity parental level of education      lunch \
0  female      group B      bachelor's degree      standard
1  female      group C      some college      standard
2  female      group B      master's degree      standard
3   male      group A      associate's degree  free/reduced
4   male      group C      some college      standard

```

```

  test preparation course  math score  reading score  writing score
0              none        72          72          74
1      completed        69          90          88
2              none        90          95          93
3              none        47          57          44
4              none        76          78          75

```

Multiple Linear Regression Results:

```

MAE : 4.214763142474852
MSE : 29.09516986671551
R2  : 0.8804332983749564

```

Ridge Regression Results:

```

MAE : 4.214216440671403
MSE : 29.121240871114434
R2  : 0.8803261594918252

```

Lasso Regression Results:

```

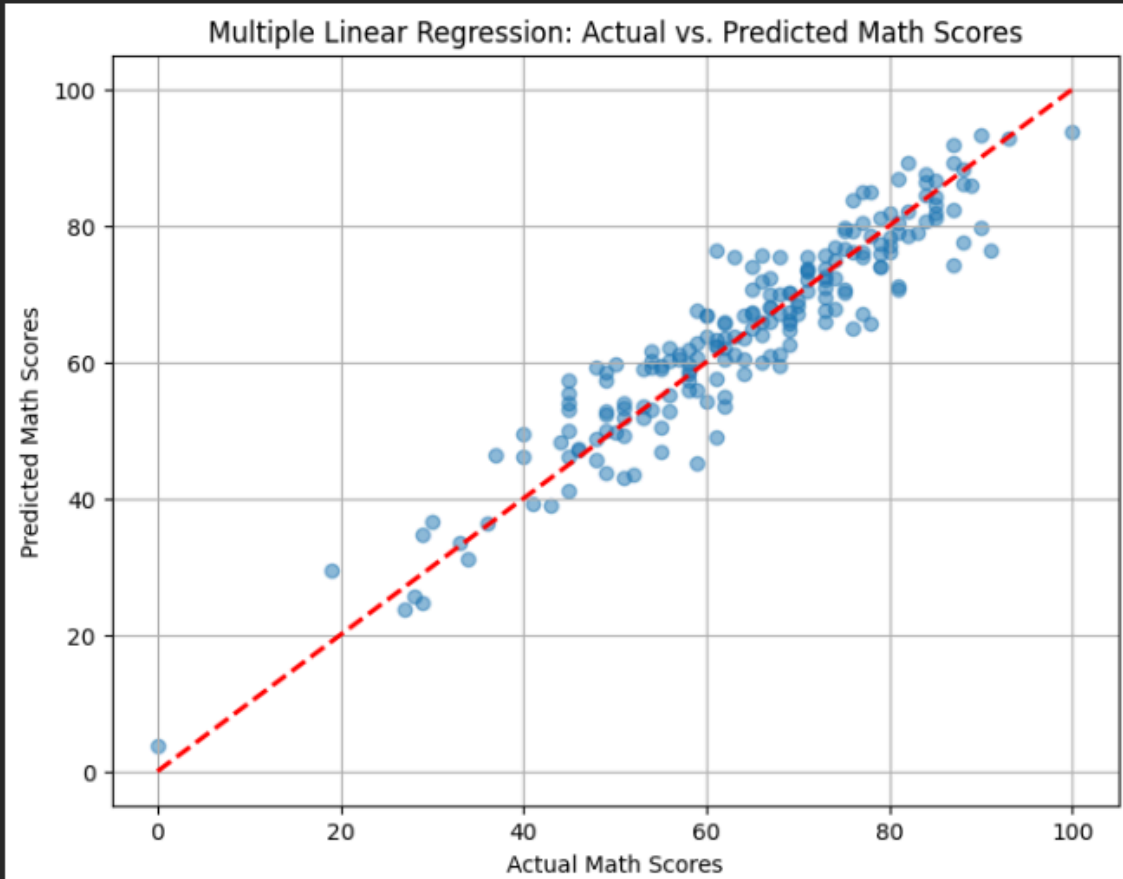
MAE : 4.14853142437036
MSE : 28.699072368950066
R2  : 0.8820610624178097

```

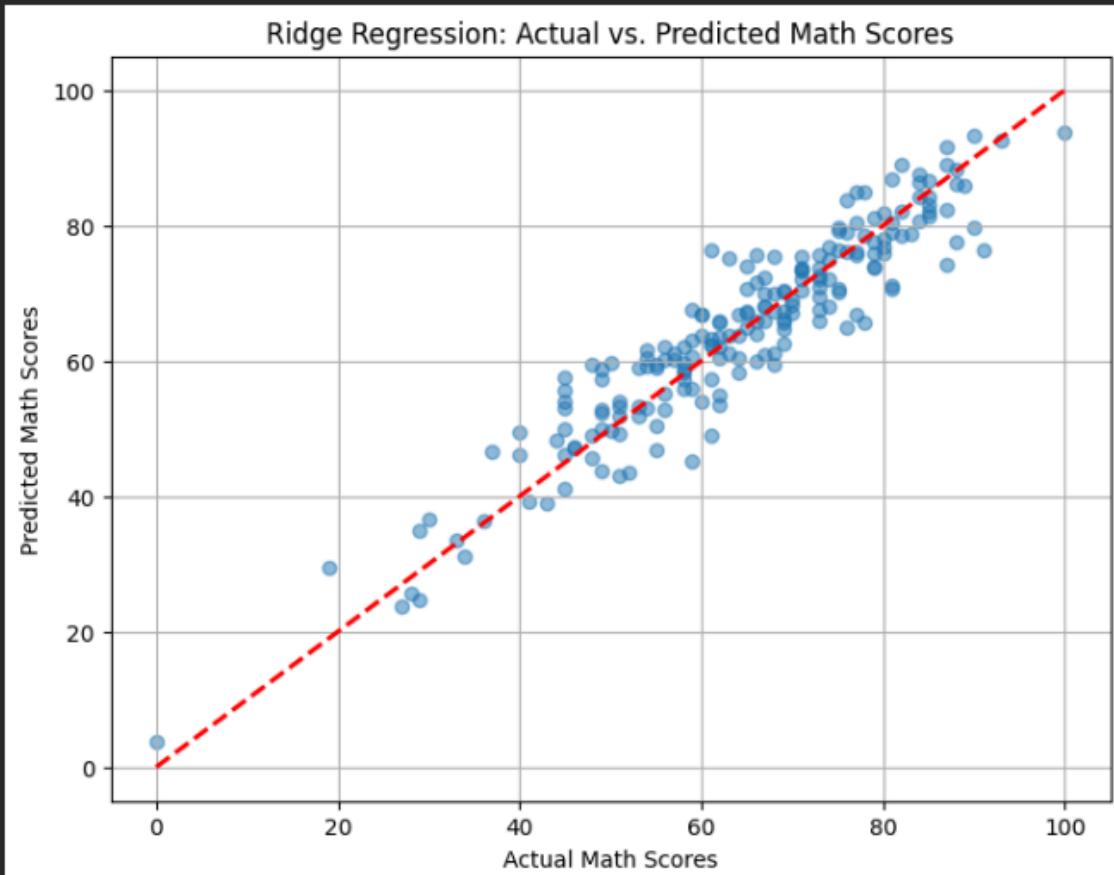


```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_linear, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Multiple Linear Regression: Actual vs. Predicted Math Scores')
plt.xlabel('Actual Math Scores')
plt.ylabel('Predicted Math Scores')
plt.grid(True)
plt.show()
```



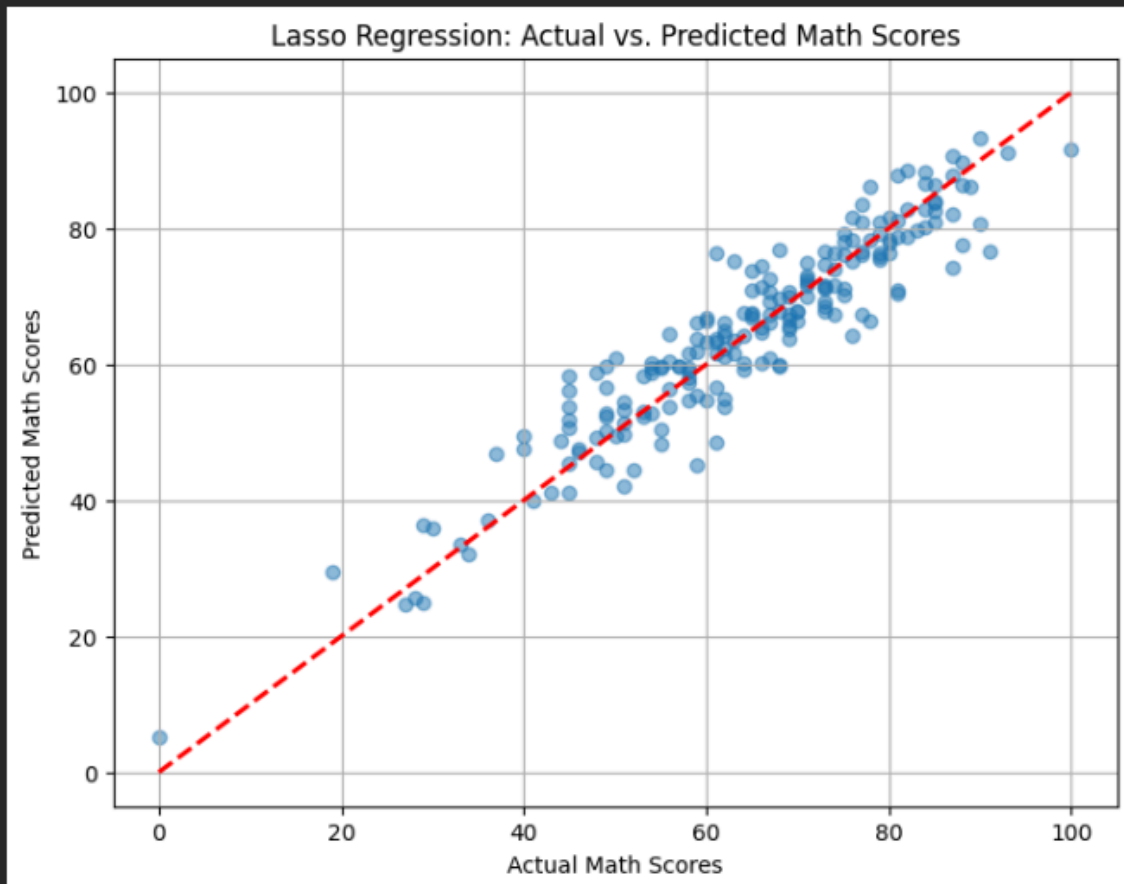
```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_ridge, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Ridge Regression: Actual vs. Predicted Math Scores')
plt.xlabel('Actual Math Scores')
plt.ylabel('Predicted Math Scores')
plt.grid(True)
plt.show()
```



```

plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_lasso, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Lasso Regression: Actual vs. Predicted Math Scores')
plt.xlabel('Actual Math Scores')
plt.ylabel('Predicted Math Scores')
plt.grid(True)
plt.show()

```



Conclusion:

Multiple Regression provides baseline prediction performance.

Ridge Regression improves stability and handles multicollinearity.

Lasso Regression performs feature selection and simplifies the model.

Regularization techniques enhance model robustness and prevent overfitting.